

Esame OS Internals (Cabodi)

Data: 25/06/2024

1 SECTION 1: THEORETICAL (Silberschatz)

Domanda 1 (Politiche di Sostituzione e Working Set)

Q1-Q3) Working Set:

1. La politica di sostituzione pagine detta "working set" usa un resident set a dimensione fissa (TRUE/FALSE). Fornire una breve motivazione.
2. Perché è difficile implementarla (la politica working set)? (scegliere la/e risposta/e giusta/e, sono possibili più risposte):
 - a. Perché la tecnica richiede di tenere, per ogni pagina, una lista dei tempi di accesso
 - b. Perché il resident set deve essere aggiornato anche in caso di assenza di PF
 - c. Perché la tecnica deve tenere traccia del tempo di ultimo accesso per ogni pagina
 - d. Perché è difficile indovinare un tempo delta adeguato

Q4) PT vs IPT (Calcoli dimensionali):

Consider a process having a virtual address space of 48 GB, in a system equipped with 16GB of RAM, with 64-bit architecture (Byte addressing) and paged memory management (4KB pages/frames) for user processes only, so not for the kernel. Assume 4GB of RAM is statically allocated to the kernel. You want to compare a standard page table-based solution (one table for each process) and an IPT-based solution. Let's assume that the pid of a process is represented on 10 bits, and that 4 extra bits (e.g valid, dirty, etc.) are needed for each entry (for both PT and IPT). Let's call x the number of bits needed for page and/or frame indexes (for both PT and IPT). Calculate:

- A) The lower bound (minimum possible value) for x , and the size of the PT (single level) for the process and of the IPT (use for x the lower bound value). Take into account that, due to alignment, an IPT or PT entry can only be 32 or 64 bits wide.
- A) Finally, using the proposed IPT, say what is the maximum possible size for the virtual address space of a process.

Q5) Algoritmo LRU a Stack:

Si consideri una strategia LRU basata su un'implementazione a stack (con 5 frame). Data la seguente stringa di riferimento: 4, 6, 4, 1, 7, 8, 2, 2, 3 (T1), 4, 2 (T2).

Mostrare lo stack al tempo T1 (dopo aver effettuato l'accesso alla pagina 3) e T2 (dopo aver effettuato l'accesso alla pagina 2). Rappresentare lo stack con numeri separati da virgole, con lo stack top per primo (ad es. 1,2,3,4,5 significa che 1 è in cima allo stack).

Q6-Q8) Page Faults Probabilities:

Si considerino due stringhe di riferimento $w1$ e $w2$ di uguale lunghezza. Sappiamo che $p2 = 2 \cdot p1$

(probabilità delle stringhe). Vengono valutati due algoritmi di sostituzione delle pagine: A1 e A2. A1 produce lo stesso numero di page fault ($F1 = F2 = 5000$) con $w1$ e $w2$. Nel caso di A2, $F1$ e $F2$ cambiano, ma la loro somma rimane invariata. Poiché la frequenza PF con A1 è peggiore del 20% rispetto ad A2, alla fine viene selezionato A2. Calcolare i valori di $F1$ e $F2$ misurati con l'algoritmo A2. Mostrare i passaggi intermedi.

Domanda 2 (File System, Blocchi e Frammentazione)

Q9) Esercizio File Binario:

Dato un file binario di dimensioni 23033 KB, calcolare esattamente il numero di blocchi di indice e blocchi di dati occupati dal file. Calcolare anche la frammentazione interna, sia per i blocchi di dati che per i blocchi di indice.

Q10-Q11) Esercizio Record a Lunghezza Variabile:

Un file di testo B, di dimensione 15300 B, contiene una sequenza di righe di lunghezza variabile, ciascuna terminata con `'\n'`. Sappiamo che la lunghezza media della riga è di 50 caratteri (`'\n'` escluso) e la lunghezza massima della riga è 100. Calcolare il numero di righe nel file (nel caso in cui il numero sia compreso tra un valore minimo e un valore massimo, calcolare l'intervallo). Consideriamo il file B appena calcolato:

- Il formato di record a lunghezza variabile influisce sull'allocazione?
- Tutti i blocchi allocati memorizzano la stessa quantità di righe di file, data dalla dimensione di un blocco divisa per la lunghezza massima della riga?

Q12) Esercizio Layout Disco e Bitmap:

A given disk is organized with physical and logical blocks of size 8KB. The disk contains multiple partitions: partition A, of NB blocks, is formatted for a file system that statically allocates NM blocks for metadata (that include directories, file control blocks and a bitmap, for free space management), and ND blocks for file data. The bitmap has one bit for each of the ND data blocks. $NM/4$ of the metadata blocks are reserved to the bitmap. Answer the following questions:

- A) Compute the ratio ND/NM .
 - A) Suppose the bitmap indicates a 33.33% ratio free/used blocks (so 1 free block for 3 used), compute (as a function of NM) the maximum size for a contiguous interval of free blocks, assuming the most favorable bitmap configuration (favorable means able to obtain larger free partitions). Give the same answer when assuming the less favorable bitmap configuration.
-

Domanda 3 (Paginazione, TLB e Slab Allocator)

Q13-Q14) Gestione Memoria (Rispondere SI/NO con breve motivazione):

Prendere in considerazione la gestione della memoria con paginazione e una MMU con un TLB. Per ciascuna delle seguenti domande, rispondere SI o NO e motivare:

- La frammentazione aumenta quando aumentano le dimensioni della pagina, perché sono necessarie partizioni contigue più grandi?
- Una free list di pagine ha una frammentazione interna media di mezza pagina?
- Tutte le strutture dati del kernel richiedono un'allocazione contigua?
- L'allocatore "slab" utilizza solo dimensioni potenza di 2?

- Il prepagging è vantaggioso solo se la probabilità che una pagina prepaginata venga utilizzata in modo efficace è $>$ dell'80%?
- La copertura della TLB (TLB reach) diminuisce all'aumentare delle dimensioni della pagina?

2 SECTION 2: OS161

Domanda 4 (Indirizzi, Configurazione e dumbvm)

Q15) Classificazione Indirizzi:

Per ciascuno dei seguenti indirizzi, dire se possono essere un indirizzo utente logico, un indirizzo logico del kernel, o un indirizzo fisico:

- 0x80803005 (User Logico / Kernel Logico / Fisico)
- 0x312010 (User Logico / Kernel Logico / Fisico)
- 0x532100 (User Logico / Kernel Logico / Fisico)

Q16) Configurazione `conf.kern`:

An OS161 system is given. Suppose you have added the following instructions to `kern/conf/conf.kern`:
`defoption project`

`optfile project syscall/project.c`

and you have created the `PROJECT` file in `kern/conf`, copied from the `DUMBVM` file.

- B) Tell if the actions described above, plus the execution (in `kern/conf`) of `./config PROJECT`, are sufficient for the optional file `syscall/project.c` to be compiled when you execute (in `kern/compile/PROJECT`) the commands `bmake depend` and `bmake`.
- B) Which one between the `project.h` and `opt-project.h` files is automatically generated by the command `./config PROJECT`? Is it always generated, or only if the following instruction appears in `PROJECT`? (`options project`)
- B) What does the file contain (the automatically generated one, referred by the previous question)?

Q17-Q18) Traduzione Indirizzo OS161:

Dato l'indirizzo logico user 0x4010, convertirlo nell'indirizzo fisico correlato. È noto che `as->as_pbase1`, `as->as_pbase2`, `as->as_vbase1`, `as->as_vbase2`, `as->as_npages1`, `as->as_npages2` hanno i seguenti valori: 0x100000, 0x200000, 0x3000, 0x6000, 2, 4. La risposta va data in forma esadecimale (0x...). Fornire formule e/o passaggi intermedi.

Q19-Q20) Funzione `ram_stealmem`:

Physical memory in `dumbvm` is allocated by calling function `ram_stealmem`. For each of the following sentences, answer true/false (provide a short explanation):

- The allocation scheme used by `ram_stealmem` will not allow interleaving of kernel and user physical RAM contiguous partitions (in other words kernel partitions and user partitions will never be interleaved).
- When using a bitmap in order to keep track of allocated/free RAM, a second array storing the sizes of allocated partitions is required for allocation of both kernel dynamic memory and user processes.
- The stack of a user process is not allocated (in `dumbvm`) calling `ram_stealmem`, because the size of the stack is a constant, and the stack is always located at the same logical addresses.
- Though the solution proposed in order to support memory de-allocation (`freeppages`) is based on a bitmap, a linked list could be used as well.
- Function `ram_stealmem` incrementally allocates memory starting at the first available address after loading the kernel.
- Function `ram_stealmem` is called for memory allocation of both kernel (dynamic memory) and user processes.

Domanda 5 (Syscalls, Trapframe e Locks)

Q21) Funzione `sys_getpid` (True/False):

Consider function `sys_getpid` (prototype: `pid_t sys_getpid(void);`) implemented in lab4. For each of the following sentences, answer true/false:

- The function is needed in order to implement the `getpid` system call, and it will be directly called by a user process.
- The function is called by `sys_exit`, in order to know the pid of the process to be signalled.
- The function returns the pid of the current process.
- The function is called to obtain the pid of another process (the one to be waited for termination).
- The function is needed in order to implement `sys_waitpid`, because the pid of the waiting process is needed.
- The function is needed in order to implement the `getpid` system call, and it will be called by the kernel, in function `syscall`.

Q22/23) Architettura Processi OS161:

Consider a user process on OS161 (for all questions provide an answer and a motivation/explanation).

- B) Which one, between `trapframe` and `switchframe`, is used to handle a system call?
- B) Is the `trapframe` allocated in the user stack or the kernel-level process stack? (motivate)
- B) Why are I/O devices mapped to `kseg1`? Could an I/O device be mapped to `kseg0`? (motivate the answer)

Q24-Q26) Implementazione Lock (Scegliere opzioni corrette con motivazione):

- Il lock può essere implementato da: 1) Un semaforo binario, più un elemento/requisito aggiuntivo. 2) Da un semaforo binario, senza alcun altro elemento/requisito. 3) Da un wait channel. 4) Da una condition variable.
- L'implementazione della funzione `lock_acquire` può includere una chiamata a: 1) Le funzioni `spinlock_data_get` e `spinlock_data_testandset`. 2) La funzione `wchan_sleep`. 3) La funzione `cv_wait`. 4) La funzione `P` su un semaforo.